RAPPORT DE PROJET DE FIN D'ANNÉE

Workflow

Plateforme de gestion du travail en équipe, avec une intelligence artificielle pour trier les tickets

RÉALISÉ PAR

Hamza ZIOUANE
Abdelmouniem OUADOUDI
Ilyas LABHEL

ENCADRÉ PAR

Dr Youness TABII

Année universitaire 2025 – 2026

Dédicace

À nos parents, pour leur soutien et leur patience.

À nos familles et à nos amis, pour leurs encouragements.

À tous nos professeurs de l'ISGA.

Remerciements

Nous remercions d'abord notre encadrant, **Dr Youness TABII**, pour son temps, ses conseils et ses remarques pendant tout le projet.

Nous remercions aussi la direction et tous les professeurs de l'ISGA pour la formation qu'ils nous donnent.

Enfin, nous remercions les membres du jury, qui ont accepté d'évaluer notre travail.

Résumé

Ce rapport présente **Workflow**, notre projet de fin d'année. Workflow est une application web pour gérer le travail d'une équipe, comme Jira. On peut créer des projets, des tableaux Kanban et Scrum, des sprints et des tickets.

Notre application ajoute une intelligence artificielle. Quand un utilisateur crée un ticket, l'IA propose un type, une priorité, une équipe et un effort. Elle donne aussi un score de confiance. Elle cherche les tickets qui se ressemblent, pour éviter les doublons.

L'application est construite en microservices avec Spring Boot. Elle utilise RabbitMQ pour les messages, PostgreSQL avec pgvector pour les données, et l'API Groq pour l'IA. L'interface est faite avec React.

Mots-clés : gestion de projet, tickets, microservices, Spring Boot, React, intelligence artificielle, RabbitMQ, JWT.

Abstract

This report presents **Workflow**, our end-of-year project. Workflow is a web application for team work management, similar to Jira. Users create projects, Kanban and Scrum boards, sprints and issues.

The application adds an artificial intelligence layer. When an issue is created, the AI suggests a type, a priority, a team and an effort, with a confidence score. It also finds similar issues to avoid duplicates.

The system is built as microservices with Spring Boot. It uses RabbitMQ for messaging, PostgreSQL with pgvector for storage, and the Groq API for AI. The user interface is built with React.

Keywords: project management, issues, microservices, Spring Boot, React, artificial intelligence, RabbitMQ, JWT.

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface (interface de programmation)
DLQ	Dead Letter Queue (file des messages en erreur)
IA	Intelligence artificielle
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token (jeton de connexion)
MCD	Modèle conceptuel de données (méthode Merise)
PFA	Projet de fin d'année
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language

Table des matières

Introduction générale

Chapitre 1 — Contexte général du projet

- 1.1 Présentation de l'établissement
- 1.2 Problématique
- 1.3 Objectifs du projet
- 1.4 Démarche de travail et planning

Chapitre 2 — Analyse des besoins

- 2.1 Les acteurs
- 2.2 Besoins fonctionnels
- 2.3 Besoins non fonctionnels

Chapitre 3 — Conception

- 3.1 Diagramme de classes
- 3.2 Modèle conceptuel de données
- 3.3 Diagramme de séquence

Chapitre 4 — Architecture et technologies

- 4.1 Pourquoi des microservices ?
- 4.2 Les quatre services
- 4.3 Les trois bases de données
- 4.4 Quand quelque chose ne marche pas
- 4.5 Outils et technologies

Chapitre 5 — Réalisation

- 5.1 La couche d'intelligence artificielle
- 5.2 La sécurité
- 5.3 Les interfaces

Conclusion générale et perspectives

Webographie

Liste des figures et des tableaux

Figure 1 : Diagramme de classes UML

Figure 2 : Modèle conceptuel de données (Merise)

Figure 3 : Diagramme de séquence — création d'un ticket et classification par l'IA

Figure 4 : Page de connexion

Figure 5 : Page des projets

Figure 6 : Tableau Scrum

Figure 7 : Page de détail d'un ticket

Figure 8 : Page de profil

Tableau 1 : Les étapes du projet

Tableau 2 : Les acteurs du système

Tableau 3 : Les quatre services

Tableau 4 : Les trois bases de données

Tableau 5 : Outils et technologies

Tableau 6 : Ce que l'IA propose

Tableau 7 : Test du seuil de ressemblance

Tableau 8 : Les deux niveaux de rôle

Introduction générale

Aujourd'hui, presque toutes les équipes informatiques utilisent un outil pour suivre leur travail. Chaque tâche, chaque bug et chaque demande devient un *ticket*. Ces outils sont utiles, mais ils ont un problème : il y a beaucoup de tickets, et quelqu'un doit les trier à la main.

Dans ce projet, nous avons créé **Workflow**. C'est une application de gestion du travail, avec une intelligence artificielle qui aide à trier les tickets. L'IA propose des informations, mais c'est toujours l'humain qui décide.

Ce rapport a cinq chapitres :

- Le **chapitre 1** présente le contexte, le problème et nos objectifs.
- Le **chapitre 2** décrit les acteurs et les besoins.
- Le **chapitre 3** présente la conception : les diagrammes UML et Merise.
- Le **chapitre 4** explique l'architecture et les technologies.
- Le **chapitre 5** montre la réalisation : l'IA, la sécurité et les interfaces.

Nous terminons par une conclusion et des perspectives.

CHAPITRE 1

Contexte général du projet

Dans ce chapitre, nous présentons notre établissement, le problème que nous voulons résoudre, nos objectifs et notre méthode de travail.

1.1 Présentation de l'établissement

L'ISGA (Institut Supérieur du Génie Appliqué) est une école privée d'enseignement supérieur au Maroc. Elle fait partie du groupe **Edvantis Higher Education Group**, créé en 1981. L'école forme des ingénieurs et des cadres en informatique et en management.

Ce projet est notre projet de fin d'année (PFA). Il nous permet d'utiliser dans un vrai projet ce que nous avons appris en cours.

1.2 Problématique

Les outils de tickets classiques ont trois problèmes :

- Les équipes créent beaucoup de tickets. Personne n'a le temps de les trier : le type, la priorité et l'équipe sont souvent oubliés.
- Le même problème est souvent signalé deux ou trois fois, par des personnes différentes.
- Beaucoup de tickets arrivent incomplets : pas de version, pas d'étapes, pas de capture d'écran.

La question de notre projet est donc : *comment aider une équipe à trier ses tickets automatiquement, sans perdre le contrôle humain ?*

1.3 Objectifs du projet

- Créer un outil de gestion du travail complet : projets, tableaux Kanban et Scrum, sprints, tickets, commentaires et fichiers.
- Ajouter une IA qui propose le type, la priorité, l'équipe et l'effort de chaque ticket.
- Détecter les tickets qui se ressemblent, pour éviter les doublons.
- Signaler les informations qui manquent dans un ticket.
- Protéger l'application : connexion, rôles et droits par projet.
- Construire une architecture en microservices.

Notre règle principale est simple : **l'IA propose, l'humain décide.**

1.4 Démarche de travail et planning

Nous avons travaillé par étapes (jalons). Chaque étape ajoute une seule chose. À la fin de chaque étape, l'application marche et on peut faire une démo. Nous avons commencé avec une seule application simple, et nous l'avons divisée en services seulement quand c'était nécessaire.

Étape	Nom	Ce que nous avons ajouté
M0	Démarrage	La structure du projet, la base de données PostgreSQL avec Docker, et un premier service qui démarre.
M1	Tickets	Les projets, les tableaux, les sprints et les tickets. Le tableau Kanban en React, avec le glisser-déposer.
M2	Connexion	Le service d'authentification, le jeton JWT, la passerelle et l'annuaire Eureka. L'application devient microservices.
M3	IA	RabbitMQ et le service de classification, qui appelle l'API Groq.
M4	Doublons	La recherche de tickets semblables avec pgvector, et un tableau de bord.
M5	Rôles	La validation des comptes par un administrateur, et les rôles par projet.

Tableau 1 : Les étapes du projet

Dans ce chapitre, nous avons présenté le contexte du projet. Le chapitre suivant décrit les besoins.

CHAPITRE 2

Analyse des besoins

Dans ce chapitre, nous présentons les utilisateurs du système et ce qu'ils doivent pouvoir faire.

2.1 Les acteurs

Acteur	Rôle
Administrateur (ADMIN)	Il valide les nouveaux comptes et choisit leur rôle. Il peut relancer les messages en erreur.
Manager (MANAGER)	Il peut créer un projet. Il devient alors chef de ce projet.
Développeur (DEVELOPER)	Il rejoint des projets et travaille sur les tickets.
Chef de projet	Il gère les membres de son projet : il accepte ou refuse les demandes.
Membre du projet	Il crée des tickets, les déplace sur le tableau et écrit des commentaires.
Service d'IA (Groq)	Système externe. Il analyse le texte des tickets et renvoie des propositions.

Tableau 2 : Les acteurs du système

2.2 Besoins fonctionnels

Gestion des comptes

- S'inscrire. Le compte reste en attente jusqu'à la validation par un administrateur.
- Se connecter et se déconnecter.
- Voir et modifier son profil.

Gestion des projets

- Créer un projet. Le créateur devient chef de projet.
- Rejoindre un projet avec un code secret.
- Accepter ou refuser une demande pour rejoindre le projet.

Gestion du travail

- Créer des tableaux Kanban ou Scrum.
- Créer, démarrer et terminer des sprints. Un seul sprint peut être actif par tableau.
- Créer, modifier et supprimer des tickets.

- Déplacer un ticket sur le tableau pour changer son statut (À faire, En cours, Terminé).
- Ajouter des commentaires et des fichiers joints.
- Voir l'historique des changements de statut.

Intelligence artificielle

- Recevoir une proposition de type, de priorité, d'équipe et d'effort pour chaque nouveau ticket.
- Accepter ou corriger cette proposition.
- Voir les tickets semblables avant de créer un nouveau ticket.
- Voir la liste des informations qui manquent.

2.3 Besoins non fonctionnels

- **Rapidité :** le ticket est créé tout de suite. L'utilisateur n'attend pas la réponse de l'IA.
- **Fiabilité :** si l'IA ne répond pas, l'application continue à marcher.
- **Sécurité :** chaque requête est vérifiée. Un utilisateur voit seulement ses projets.
- **Maintenabilité :** le code est organisé par fonctionnalité. Le schéma de la base est géré par des migrations.
- **Simplicité :** l'interface est claire et facile à utiliser.

CHAPITRE 3

Conception

Dans ce chapitre, nous présentons la conception du système avec trois diagrammes. Ces diagrammes correspondent au code final.

3.1 Diagramme de classes

Le diagramme de classes montre les objets principaux et leurs liens. Les classes les plus importantes sont :

- **Project** : un projet, avec une clé courte (par exemple WM).
- **Board** : un tableau Kanban ou Scrum d'un projet.
- **Sprint** : une période de travail. Il a un état : prévu, actif ou terminé.
- **Issue** : un ticket, avec une clé unique (par exemple WM-101).
- **Comment** et **Attachment** : les commentaires et les fichiers d'un ticket.
- **AppUser** : un utilisateur. Un ticket a un rapporteur et un assigné.
- **AIClassification** : la proposition de l'IA pour un ticket. Elle est gardée à part.
- **ProjectMember** : le lien entre un utilisateur et un projet, avec son rôle dans ce projet.

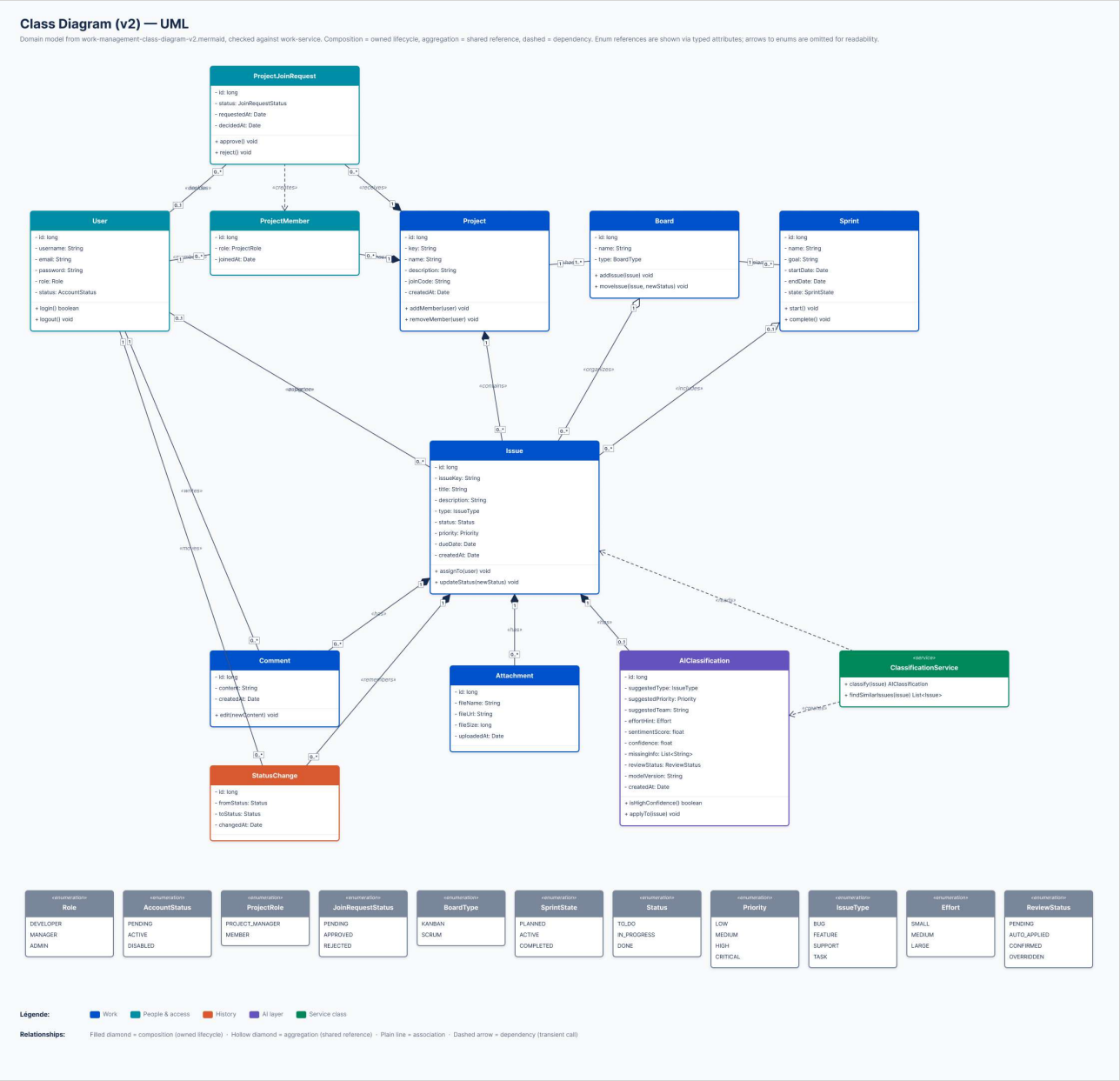


Figure 1 : Diagramme de classes UML

3.2 Modèle conceptuel de données

Le MCD présente les mêmes données avec la méthode Merise. Il montre les entités, les associations et les cardinalités. Par exemple, un projet contient plusieurs tableaux, et un tableau appartient à un seul projet.

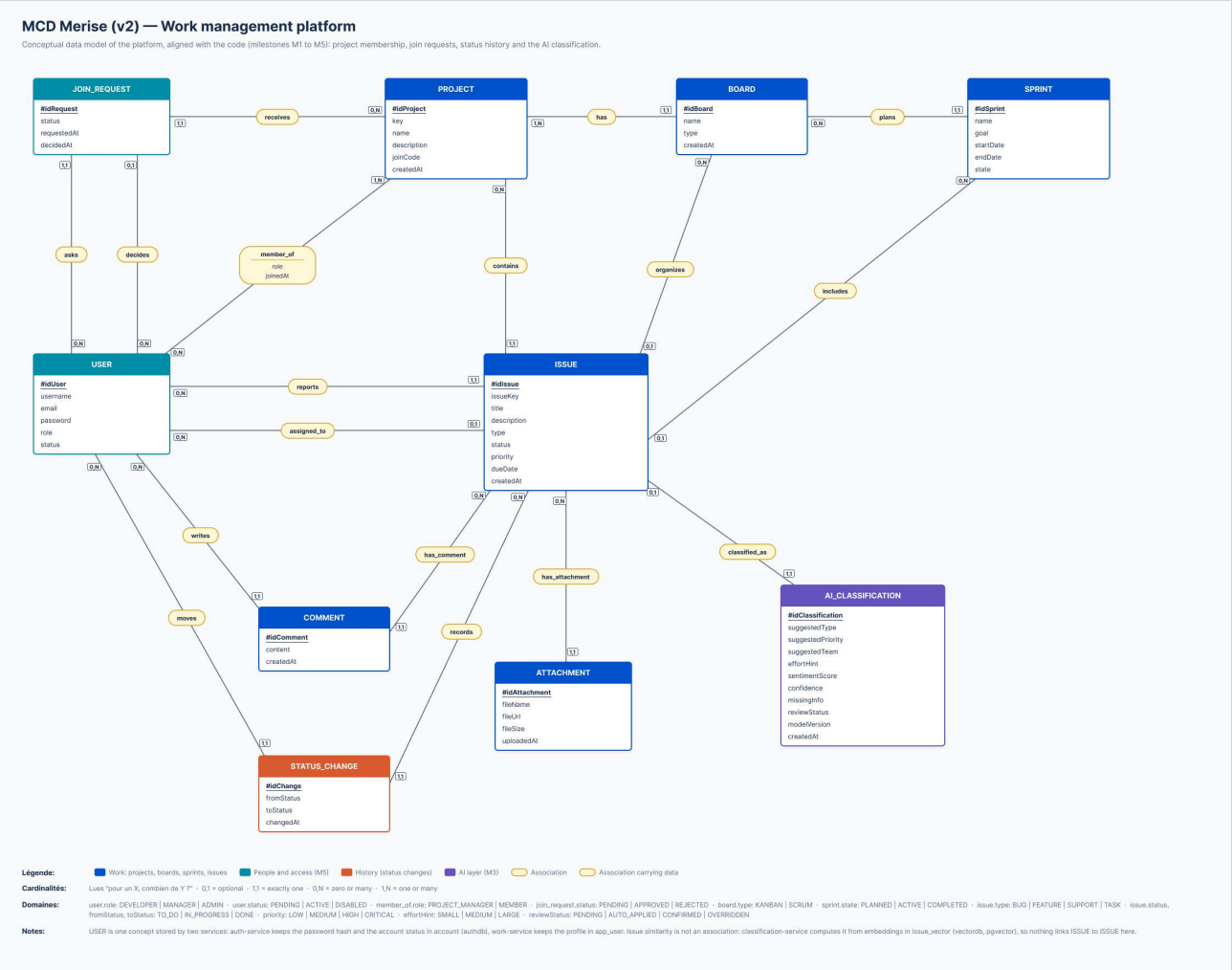


Figure 2 : Modèle conceptuel de données (Merise)

3.3 Diagramme de séquence

Ce diagramme montre ce qui se passe quand un utilisateur crée un ticket :

- L'utilisateur envoie le ticket. Le service l'enregistre et répond tout de suite.
- Le service envoie un message `issue.created` dans RabbitMQ.
- Le service de classification lit ce message et appelle l'IA Groq.
- Il renvoie le résultat avec un message `issue.classified`.
- Le service enregistre la proposition. La page se met à jour quelques secondes après.

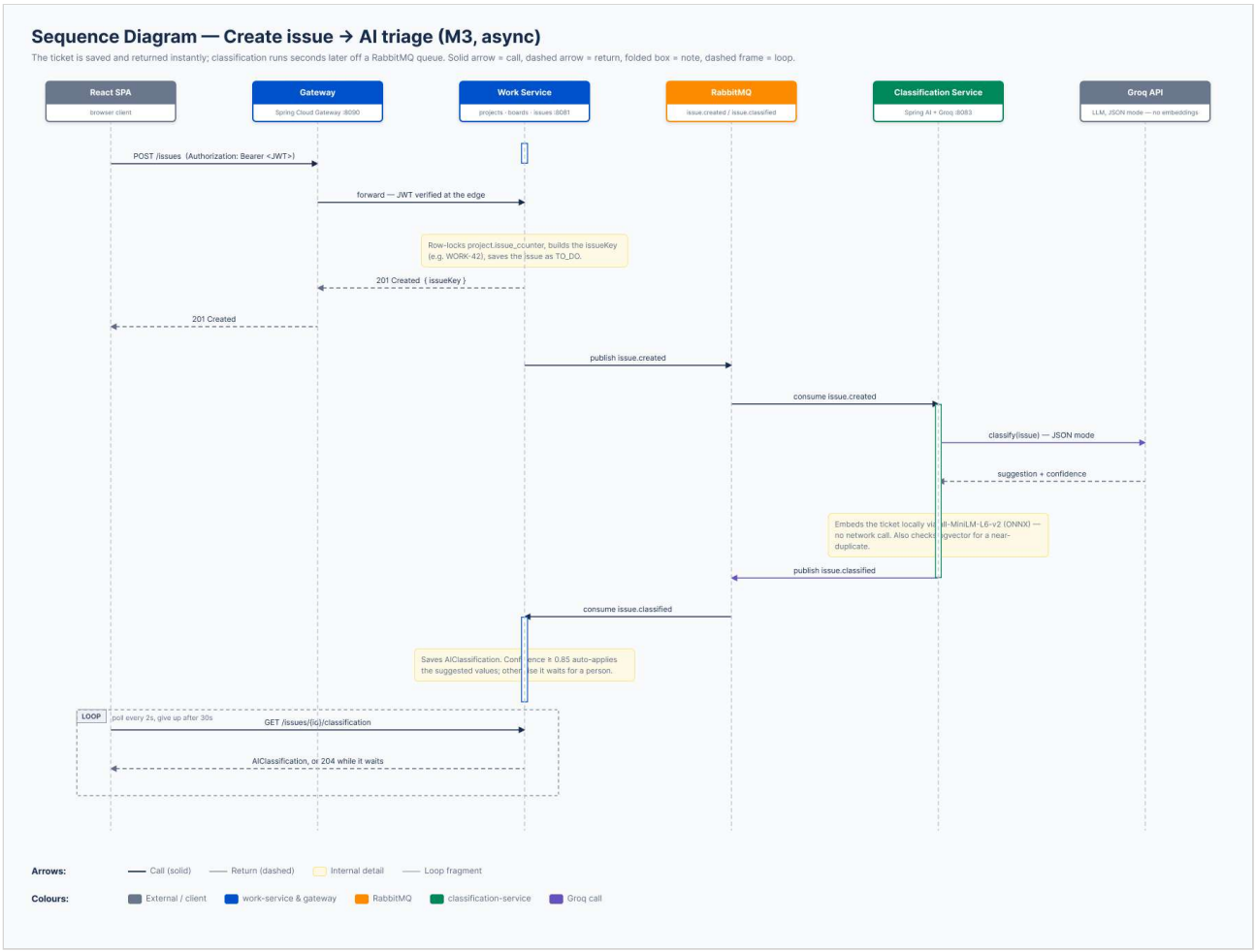


Figure 3 : Diagramme de séquence — création d'un ticket et classification par l'IA

CHAPITRE 4

Architecture et technologies

Dans ce chapitre, nous expliquons comment l'application est construite et quels outils nous avons utilisés.

4.1 Pourquoi des microservices ?

Nous avons deux raisons. D'abord, le cahier des charges demande une passerelle, un annuaire de services et des messages asynchrones. Ce n'est pas pour la performance : une seule application suffit pour quelques utilisateurs.

Ensuite, il y a un vrai avantage : l'IA est lente et elle peut tomber en panne. Elle travaille donc dans un service séparé. L'utilisateur crée son ticket tout de suite, sans attendre la réponse de l'IA.

4.2 Les quatre services

Service	Son rôle	Sa base
gateway	La porte d'entrée. Elle reçoit toutes les requêtes et vérifie le jeton.	aucune
auth-service	Les comptes et la connexion. Il crée le jeton JWT.	authdb
work-service	Les projets, les tableaux, les sprints, les tickets et les rôles.	workflow
classification-service	L'IA : il lit les nouveaux tickets et appelle Groq.	vectordb

Tableau 3 : Les quatre services

Autour de ces services, il y a **Eureka** (l'annuaire des services), **RabbitMQ** (la file de messages) et **PostgreSQL** (les bases de données).

4.3 Les trois bases de données

Base	Service	Contenu
workflow	work-service	Les données du travail : projets, tickets, commentaires… (10 tables)
authdb	auth-service	Les comptes et les mots de passe (chiffrés)
vectordb	classification-service	Un vecteur de 384 nombres par ticket, pour comparer les textes

Tableau 4 : Les trois bases de données

Chaque service a sa propre base. Un service ne peut pas lire la base d'un autre service. Pour relier un compte à son profil, nous gardons seulement l'identifiant, sans clé étrangère.

4.4 Quand quelque chose ne marche pas

- **File d'erreurs :** si un message échoue trois fois, il va dans une file spéciale. Un administrateur peut le relancer plus tard.
- **Coupe-circuit :** si un service ne répond pas, on arrête de l'appeler pendant un moment. Le reste continue à marcher.
- **Numéro de requête :** chaque requête reçoit un numéro. On peut la suivre dans tous les services.
- **Règles dans la base :** deux personnes peuvent agir en même temps. Les règles importantes sont donc vérifiées par la base de données. Par exemple, un index empêche d'avoir deux sprints actifs sur le même tableau.

4.5 Outils et technologies

Outil	Utilisation dans le projet
Java 17, Spring Boot	Le langage et le framework des services
Spring Cloud Gateway	La passerelle
Eureka	L'annuaire des services
Spring Security, JWT	La connexion et les droits
Spring Data JPA	L'accès à la base de données
Liquibase	Les migrations du schéma de la base
MapStruct	La conversion entre entités et objets de transfert (DTO)
springdoc-openapi	La documentation automatique de l'API (Swagger)
RabbitMQ	Les messages entre les services
Spring AI, Groq	L'appel au modèle d'IA
Resilience4j	Les nouvelles tentatives et le coupe-circuit
PostgreSQL 16, pgvector	Les bases de données et la recherche de textes semblables
React, Vite	L'interface utilisateur
Docker Compose	Le lancement de tous les services en local
Git, GitHub	Le suivi des versions du code

Tableau 5 : Outils et technologies

CHAPITRE 5

Réalisation

Dans ce chapitre, nous présentons le travail réalisé : l'intelligence artificielle, la sécurité et les interfaces.

5.1 La couche d'intelligence artificielle

Ce que l'IA propose

Pour chaque nouveau ticket, l'IA lit le titre et la description. Elle renvoie sept informations :

Information	Valeurs	Appliquée au ticket ?
Type	BUG, FEATURE, SUPPORT, TASK	oui, si l'IA est sûre
Priorité	LOW, MEDIUM, HIGH, CRITICAL	oui, si l'IA est sûre
Équipe	texte libre	non, seulement affichée
Effort	SMALL, MEDIUM, LARGE	non, seulement affiché
Sentiment	de −1 (fâché) à 1 (content)	non, seulement affiché
Infos manquantes	une liste	montrée à l'utilisateur
Confiance	de 0 à 1	décide si on applique

Tableau 6 : Ce que l'IA propose

La proposition est enregistrée dans une table séparée (`AIClassification`). Le ticket garde seulement la valeur choisie par l'humain. Ainsi, on peut comparer ce que l'IA a proposé et ce que l'humain a décidé.

Deux seuils

0,85 : l'IA décide seule. Si la confiance est plus grande que 0,85, l'IA change le type et la priorité du ticket toute seule. Ce seuil est haut, parce qu'une erreur change le ticket sans que personne ne la voie.

0,45 : l'IA propose un doublon. Ici, l'IA propose seulement. Une mauvaise proposition n'est pas grave : l'utilisateur la regarde et l'ignore. Ce seuil est donc plus bas.

Comment nous avons choisi 0,45

Pour trouver les doublons, chaque ticket est transformé en vecteur (une liste de 384 nombres). Deux textes proches ont des vecteurs proches. Nous avons comparé sept textes au ticket « Login page crashes with a 500 error » :

Texte testé	Score	Même problème ?
Texte identique	0,85	oui
Même panne	0,75	oui
Texte reformulé	0,72	oui
Autres mots	0,54	oui
Description vague	0,49	oui
Autre sujet	moins de 0,20	non
Texte sans sens	moins de 0,20	non

Tableau 7 : Test du seuil de ressemblance

Avec un seuil de 0,75, l'IA trouve seulement les textes presque identiques. Avec 0,45, elle trouve les cinq vrais doublons, et elle ignore les textes d'un autre sujet.

Quand l'IA ne répond pas

- Le ticket est toujours créé, même si l'IA ne répond pas.
- Sans clé Groq, un programme simple utilise des mots-clés. Le projet marche quand même.
- Si Groq est lent ou occupé, on réessaie trois fois.
- Si l'erreur est définitive, le message va directement dans la file d'erreurs.

5.2 La sécurité

Deux niveaux de rôle

Question	Réponses possibles	Où est l'information ?
Qui suis-je sur la plateforme ?	DEVELOPER, MANAGER, ADMIN	dans le jeton JWT
Qui suis-je dans ce projet ?	PROJECT_MANAGER, MEMBER	dans la base, vérifiée à chaque fois

Tableau 8 : Les deux niveaux de rôle

Une personne peut être chef dans un projet et simple membre dans un autre. Ce rôle change souvent. Il n'est donc pas dans le jeton : on le lit dans la base.

Le jeton est vérifié deux fois

D'abord, la passerelle vérifie le jeton. Une mauvaise requête s'arrête tout de suite. Ensuite, work-service vérifie aussi le jeton, car on peut l'appeler directement. Il vérifie aussi si l'utilisateur a le droit de voir ce projet. Si ce n'est pas le cas, il répond avec l'erreur 403.

L'inscription

Au début, un nouvel utilisateur pouvait choisir le rôle ADMIN pendant l'inscription. C'était une faille. Nous l'avons corrigée : maintenant, chaque compte commence **en attente**. Un administrateur valide le compte et choisit le rôle. Le premier administrateur est créé au démarrage de l'application.

Rejoindre un projet

La personne qui crée un projet devient chef de projet. Chaque projet a un code secret de 12 caractères. Le chef envoie ce code aux personnes qu'il veut inviter. Il peut changer le code à tout moment. Une personne ne peut envoyer qu'une seule demande à la fois pour le même projet.

5.3 Les interfaces

Page de connexion

L'utilisateur entre son adresse e-mail et son mot de passe. Si le compte est en attente, un message l'explique.

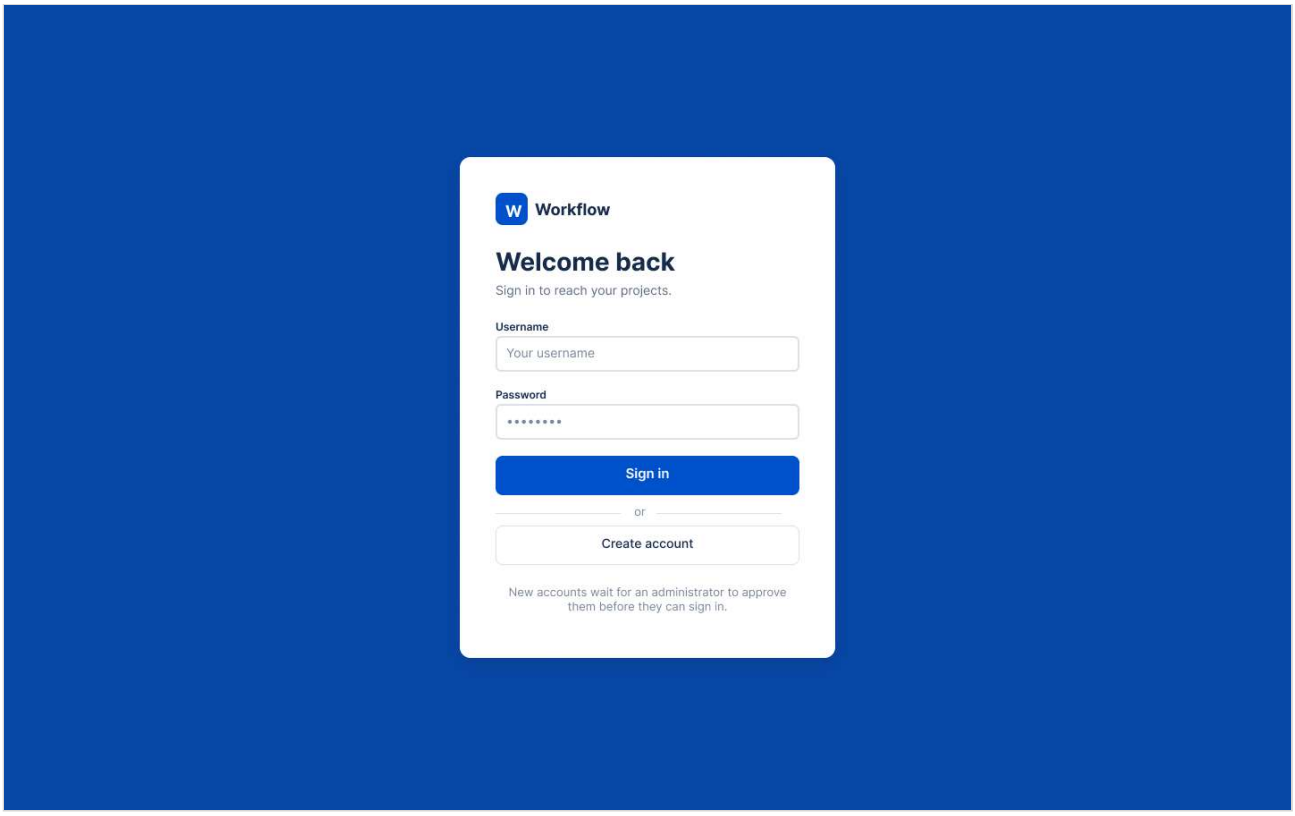


Figure 4 : Page de connexion

Page des projets

L'utilisateur voit seulement les projets dont il est membre. Il peut créer un projet ou en rejoindre un avec un code.

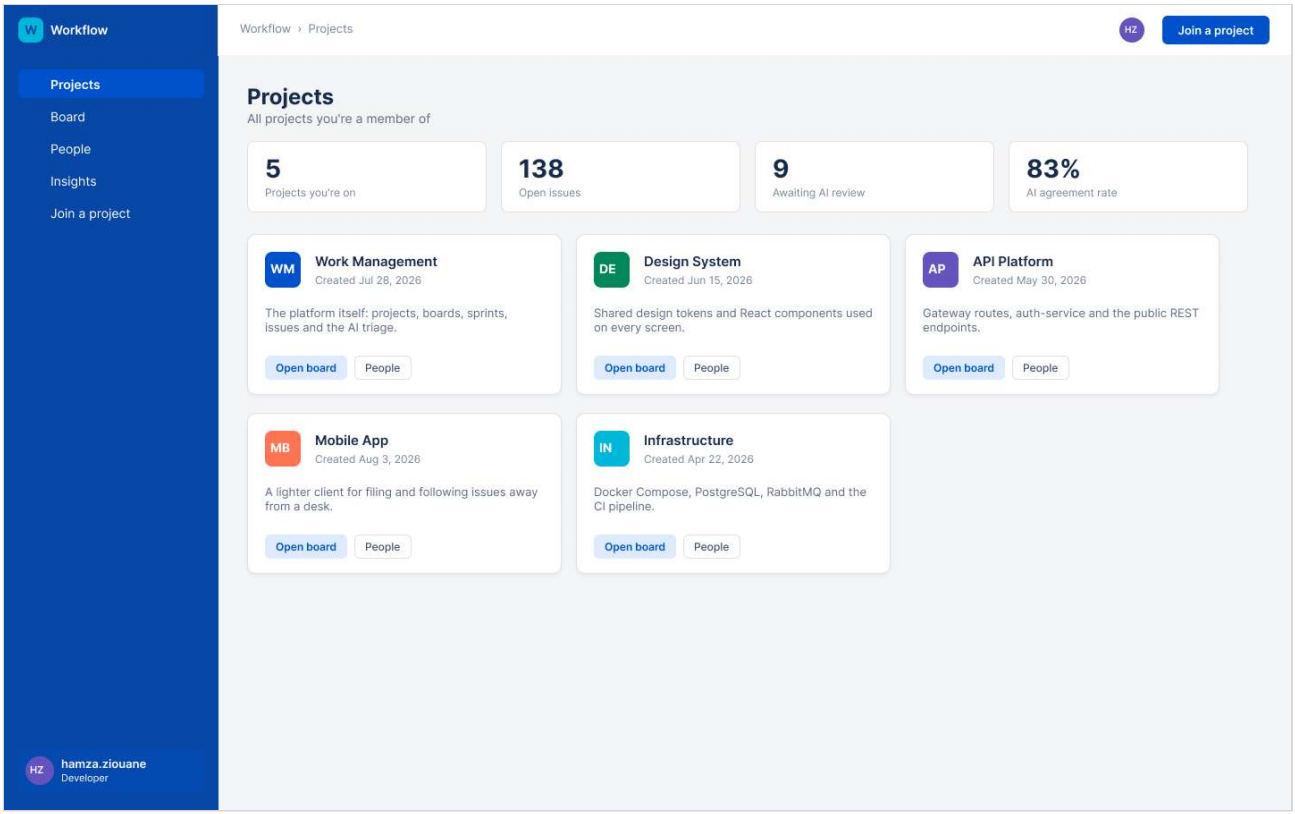


Figure 5 : Page des projets

Tableau Scrum

Chaque colonne est un statut. On déplace une carte avec la souris pour changer son statut. Pendant la création d'un ticket, l'application montre un doublon possible.

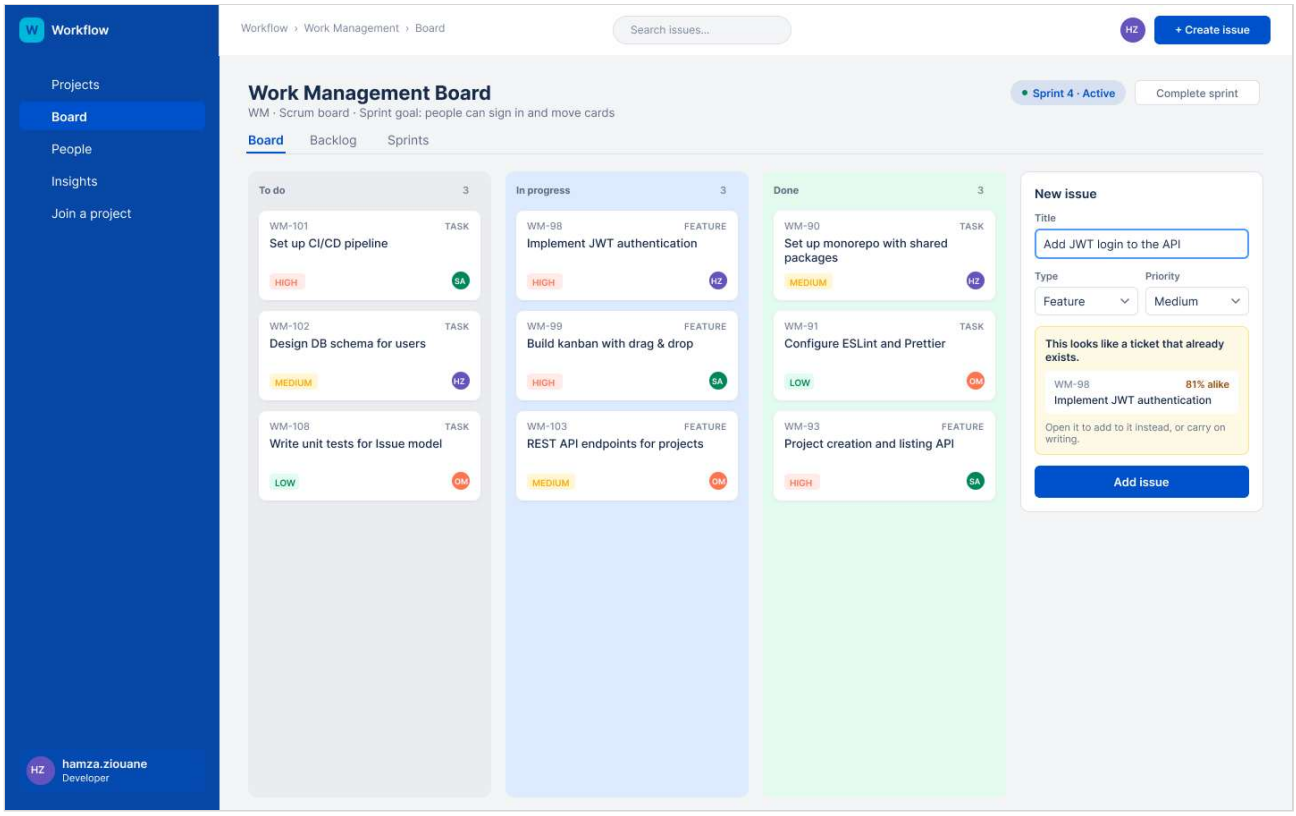


Figure 6 : Tableau Scrum

Détail d'un ticket

La page montre toutes les informations du ticket, la proposition de l'IA, les commentaires et l'historique.

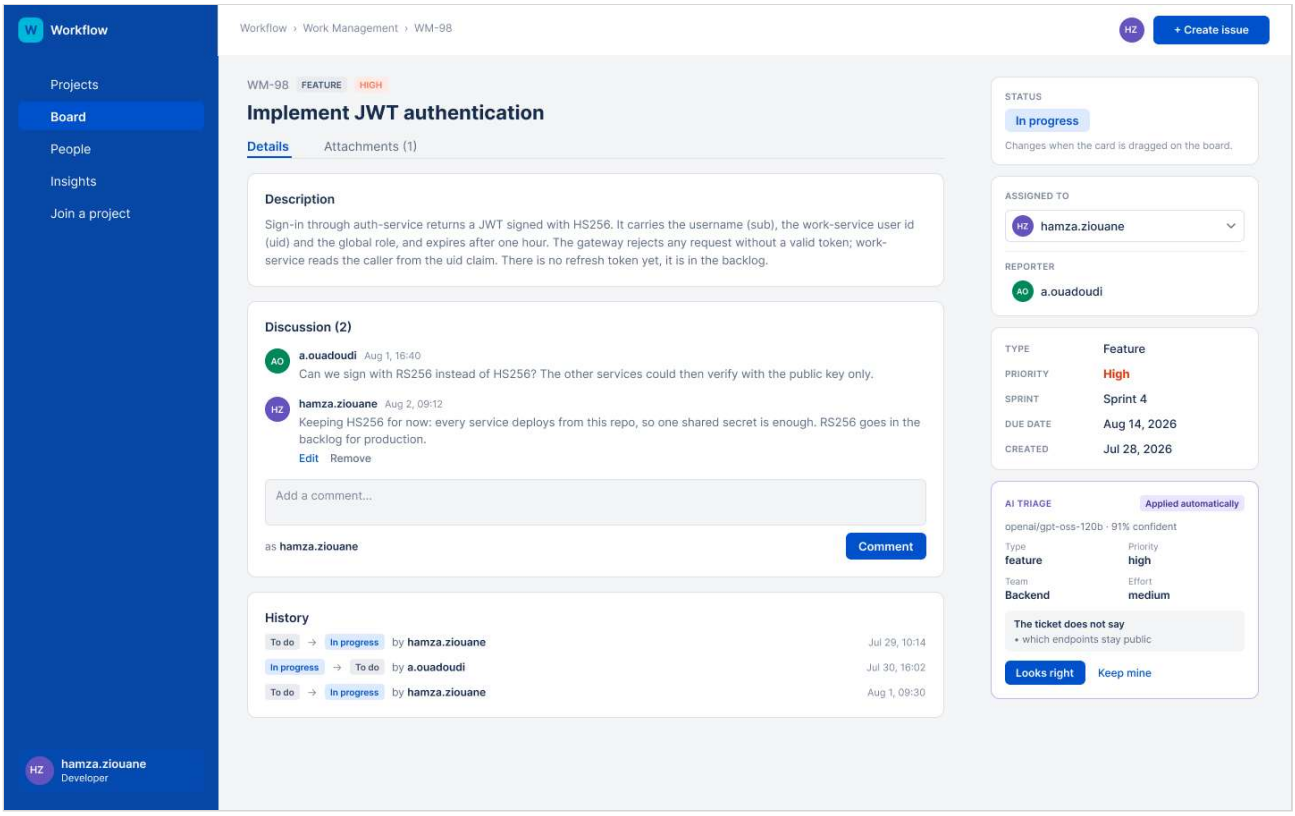


Figure 7 : Page de détail d'un ticket

Page de profil

L'utilisateur voit ses informations et son rôle sur la plateforme.

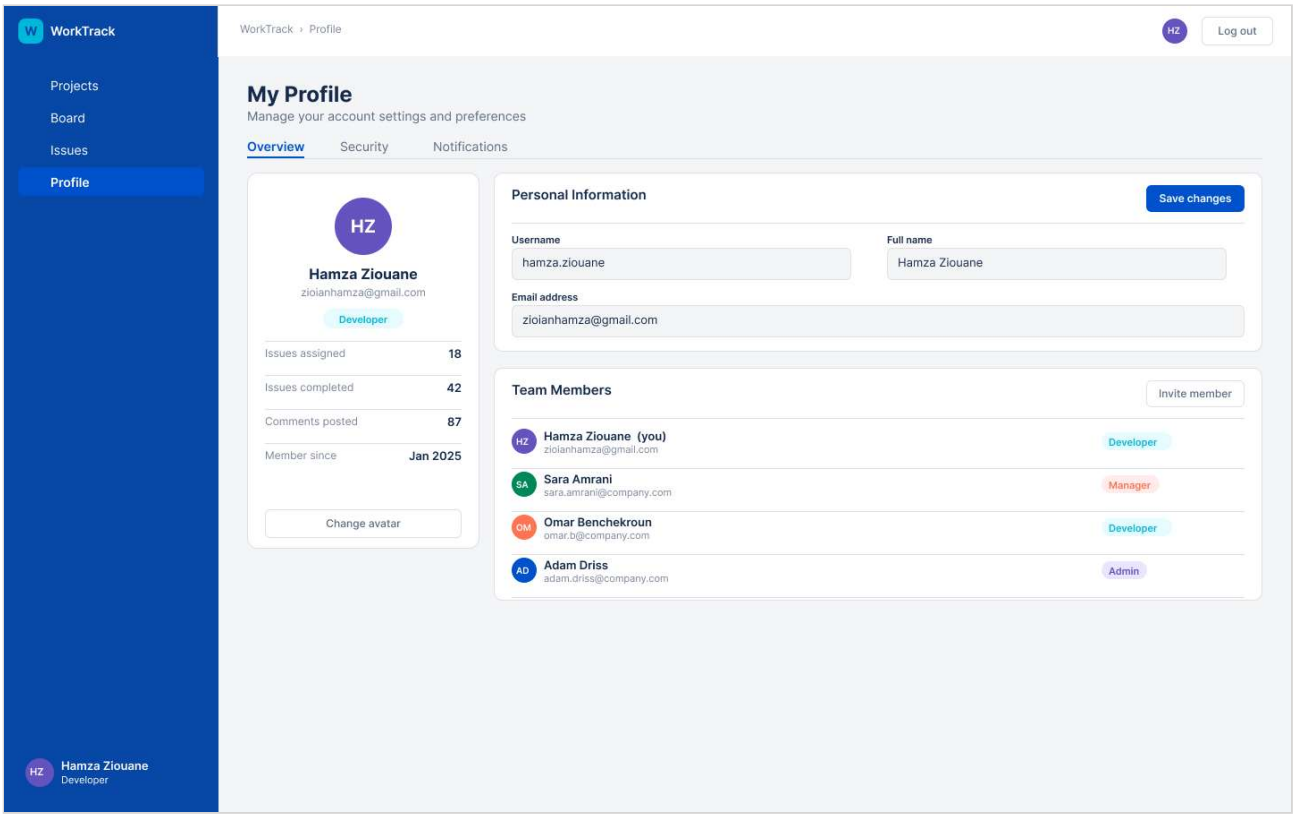


Figure 8 : Page de profil

Conclusion générale et perspectives

Dans ce projet, nous avons réalisé **Workflow**, une application de gestion du travail avec une intelligence artificielle. L'application permet de gérer des projets, des tableaux, des sprints et des tickets. L'IA propose le type, la priorité, l'équipe et l'effort de chaque ticket, et elle détecte les doublons.

Nous avons construit une architecture en microservices, avec une passerelle, un annuaire de services, une file de messages et trois bases de données. Nous avons aussi protégé l'application avec des jetons JWT et deux niveaux de rôle.

Ce projet nous a appris beaucoup de choses : travailler par étapes, faire des choix techniques et les expliquer, et construire un système qui continue à marcher quand une partie tombe en panne.

Perspectives

Plusieurs améliorations sont possibles :

- Ajouter un service de notifications (en temps réel ou par e-mail).
- Ajouter un assistant qui demande automatiquement les informations manquantes.
- Proposer une première réponse aux tickets avec l'IA, à partir des anciens tickets.
- Déployer l'application sur un serveur en ligne.

Webographie

- Spring Boot — documentation officielle : spring.io/projects/spring-boot
- Spring Cloud Gateway : spring.io/projects/spring-cloud-gateway
- Spring AI : spring.io/projects/spring-ai
- RabbitMQ — documentation : rabbitmq.com/docs
- PostgreSQL — documentation : postgresql.org/docs
- pgvector : github.com/pgvector/pgvector
- Groq — documentation de l'API : console.groq.com/docs
- Resilience4j : resilience4j.readme.io
- React : react.dev
- Docker : docs.docker.com